1)AIML Introduction

Artificial Intelligence (AI) is a branch of computer science that deals with creating machines or computer systems that can perform tasks that normally require human intelligence.

Examples

- Google Maps → route find karna
- ChatGPT → language samajhna aur answer generate karna
- Face recognition → face identify karna

The process works like this:

1. **Input:** An email arrives.
2. **Analysis:** AI checks the email's content, sender and links.
3. **Pattern Recognition:** It compares the email with patterns learned from previous emails.
4. **Prediction:** It predicts whether the email is spam or legitimate.
5. **Action:** The email is kept in the inbox or moved to the spam folder.

2)History of AI

Artificial Intelligence (AI) is the field of computer science that aims to make machines capable of reasoning, learning, problem-solving, perception, and language understanding.

1. 1940s–1950s — Foundation of AI

The idea of intelligent machines started developing **Alan Turing** made major contributions to the idea of machine intelligence.

2. 1956 — Birth of AI ⭐

- The term **Artificial Intelligence** was formally introduced by **John McCarthy**.
- At the **Dartmouth Conference (1956)**

3. 1950s–1960s — Early AI Programs

- Researchers developed programs capable of solving mathematical and logical problems.

4. 1970s — First AI Winter

- Expectations about AI became very high, but computers were not powerful enough to meet them.
- Funding and interest decreased.
- This period is known as the **AI Winter**.

5. 1980s — Expert Systems

- AI became popular again through **Expert Systems**.
- These systems used stored knowledge and rules to make decisions like human experts.
- Example: **MYCIN**

6. 2010s — Deep Learning Revolution ⭐

- Large datasets, powerful GPUs, and improved algorithms made **deep neural networks** highly successful.

- Major advances occurred in:
    - Image recognition
    - Speech recognition
    - Natural Language Processing
    - Autonomous systems

7. 2020s — Generative AI

- AI advanced rapidly in **large language models (LLMs)** and **generative AI**.
- Systems can now generate:
    - Text
    - Images
    - Audio
    - Video
    - Computer code

3)Foundations of Artificial Intelligence

1. Meaning of Foundations of AI

The foundations of Artificial Intelligence are the different fields of knowledge that provide the concepts, theories, and techniques required to develop intelligent machines.

2. Major Foundations of AI ⭐⭐⭐⭐⭐

There are 8 major foundations of AI:

1. Philosophy 🧠

Philosophy contributed ideas about:

- **Knowledge and reasoning**
- **Logic**
- **Rational thinking**
- **Mind and intelligence**
- **Decision making**

Example: How can we determine whether a conclusion is logically correct?

---

2. Mathematics ➗

Mathematics provides the formal tools required by AI.
Important areas:

- Logic
- Probability
- Statistics
- Linear algebra
- Calculus
- Optimization

Example: Machine learning uses probability and statistics to make predictions.

---

3. Economics 💰

Economics contributes concepts related to:

- Decision making
- Utility
- Preferences
- Optimization
- Game theory

AI agents often need to select an action that gives the maximum benefit/utility.
Example: A delivery AI chooses a route that minimizes cost and time.

---

4. Neuroscience 🧠

Neuroscience studies the human brain and nervous system.
It inspired AI, particularly:

- Artificial neural networks
- Learning mechanisms
- Perception

Example: ANN is inspired by the way biological neurons process information.

---

5. Psychology 🧘‍♀️

Psychology studies:

- Human learning
- Memory
- Problem solving
- Perception
- Decision making
- Human behavior

AI uses these ideas to understand and model intelligent behavior.

---

6. Computer Engineering 💻

Computer engineering provides the hardware and computational systems needed to implement AI.
It includes:

- Processors
- Memory
- Sensors
- Robots
- Computers
- Embedded systems

Example: A robot needs sensors, processors and actuators to perform intelligent tasks.

---

7. Control Theory and Cybernetics ⚙️

Control theory deals with how systems can:

- Monitor their state
- Compare it with a desired state
- Take corrective actions

It is important in:

- Robotics
- Autonomous vehicles
- Industrial automation

Example: A self-driving car continuously adjusts its steering to remain in its lane.

---

8. Linguistics 🗣️

Linguistics studies human language and its structure.
It provides the foundation for:

- Natural Language Processing (NLP)
- Speech recognition
- Machine translation
- Text analysis

Example: A chatbot needs linguistic knowledge to understand human sentences.

4)Intelligent Agent

An **Intelligent Agent** is a system that **perceives its environment through sensors and takes actions through actuators** to achieve a specific goal.
Example: A self-driving car uses cameras and sensors to observe the road and controls steering and speed to reach its destination safely.

Example:

A **robot**:

- **Sensors** → detect obstacles
- **Agent** → decides what to do
- **Actuators** → move the robot

5)Agent and Environment

The **environment** is the external world in which an agent operates.
The agent interacts with the environment using sensors and actuators.

ENVIRONMENT
↓
Sensors
↓
AGENT
↓
Actuators
↓
ENVIRONMENT

1. 6)Rationality

Rationality is the ability of an agent to choose the best possible action based on its available information and performance measure to achieve its goal.

Example

A navigation system chooses a route based on

- Distance
- Traffic
- Road conditions
- Estimated travel time

It behaves rationally if it selects the route expected to perform best according to its objective.

Factors affecting Rationality:

1. **Performance Measure** – Defines how the agent's success is evaluated.
2. **Percept Sequence** – The information the agent has received from the environment.
3. **Prior Knowledge** – Knowledge the agent already has about the environment.
4. **Available Actions** – The actions that the agent can perform.

7)Environment in Artificial Intelligence

An **environment** is the external world or surrounding in which an intelligent agent operates. The agent **perceives the environment through sensors** and **performs actions on the environment through actuators**.

Example — Self-Driving Car

ENVIRONMENT
Roads, traffic, people
↓
Sensors
↓
AGENT
↓
Actuators
↓
ENVIRONMENT

The car's **environment** includes roads, other vehicles, pedestrians, traffic lights, weather, etc. The car observes these things through cameras and sensors and then takes actions such as **steering, braking, and accelerating**.

Types of environments:

- Fully / Partially Observable
- Deterministic / Stochastic
- Episodic / Sequential
- Static / Dynamic
- Discrete / Continuous

Types of AI Environments

1. Fully Observable vs Partially Observable

Fully Observable Environment

An environment is **fully observable** when the agent's sensors provide all the relevant information needed to make a decision.
Example: Chess.

In chess, the complete board and position of all pieces are visible to both players.

> **Definition:** A fully observable environment provides the agent with complete relevant information about the current state of the environment.

Partially Observable Environment

An environment is **partially observable** when the agent cannot access all relevant information about the current state.
Example: Driving a car in fog.
The car cannot clearly see everything around it.

> **Definition:** A partially observable environment provides incomplete or limited information about the current state to the agent.

Easy Trick:

Full information → Fully Observable
Incomplete information → Partially Observable

---

2. Deterministic vs Stochastic

Deterministic Environment

In a deterministic environment, the result of an action is **completely predictable**.
Example: A chess move.
If you move a chess piece from one valid position to another, the resulting board state is predictable.

> **Definition:** A deterministic environment is one in which an action produces a predictable outcome.

Stochastic Environment

In a stochastic environment, the outcome of an action may involve **uncertainty or randomness**.
Example: Driving in real-world traffic.
Even if the driver follows the same action, another vehicle may suddenly change direction.

> **Definition:** A stochastic environment is one in which the outcome of an action cannot always be predicted with certainty.

Easy Trick:

Predictable → Deterministic
Uncertain → Stochastic

---

3. Episodic vs Sequential

Episodic Environment

In an episodic environment, each task or interaction is **independent of previous interactions**.
The current action does not significantly affect future tasks.
Example: Image classification.
If an AI classifies one image, that classification normally does not affect the classification of the next image.

> **Definition:** An episodic environment consists of independent episodes in which each decision is based mainly on the current situation.

Sequential Environment

In a sequential environment, the **current action affects future states and decisions**.
Example: Chess.
Your current move affects the opponent's possible moves and the future state of the game.

> **Definition:** A sequential environment is one in which current actions influence future states and decisions.

Easy Trick:

Independent tasks → Episodic
Current action affects future → Sequential

---

4. Static vs Dynamic

Static Environment

A static environment **does not change while the agent is deciding what action to take**.
Example: Crossword puzzle.
The puzzle remains unchanged while you think about your next answer.

> **Definition:** A static environment remains unchanged while the agent is making a decision.

Dynamic Environment

A dynamic environment **can change while the agent is making a decision**.
Example: Driving.

While the car is deciding what to do, other vehicles and pedestrians can move.

Definition: A dynamic environment can change while the agent is deciding or acting.

Easy Trick:

Doesn't change → Static
Changes with time → Dynamic

---

5. Discrete vs Continuous

Discrete Environment

A discrete environment has a **finite or countable number of distinct states, actions, or percepts**.

Example: Chess.

There are distinct board positions and a defined set of legal moves.

Definition: A discrete environment has distinct and countable states, actions, or percepts.

Continuous Environment

A continuous environment has **values that can change continuously**.

Example: Self-driving car.

Speed, steering angle, position, and time can change continuously.

Definition: A continuous environment has states, actions, or percepts that can take continuously varying values.

Complete Revision Table

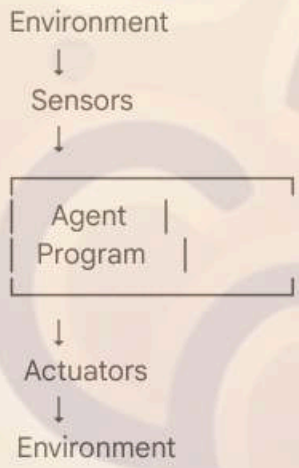
Type	Meaning	Example
Fully Observable	Complete relevant information available	Chess
Partially Observable	Information is incomplete	Driving in fog
Deterministic	Outcome is predictable	Chess
Stochastic	Outcome has uncertainty	Real-world driving

Type	Meaning	Example
Episodic	Each task is independent	Image classification
Sequential	Current action affects future	Chess
Static	Environment doesn't change during decision	Crossword
Dynamic	Environment can change during decision	Driving
Discrete	States/actions are distinct and countable	Chess
Continuous	Values change continuously	Robot movement

8)Structure of an Intelligent Agent

The **structure of an intelligent agent** describes how an agent is organized to **receive information from its environment, process that information, make decisions, and perform actions**.

Basic Structure



1. Sensors

Sensors are used by an agent to **perceive or collect information from the environment**.
Examples:

- Camera
- Microphone
- Temperature sensor
- GPS
- Touch sensor

Example: A self-driving car uses cameras and sensors to detect roads, vehicles, pedestrians, and traffic lights.

> **Definition:** Sensors are devices through which an agent receives percepts from its environment.

---

2. Agent Program

The **agent program** is the part that processes the information received from sensors and **decides what action should be taken**.
It uses:

- Current percepts
- Previous information
- Knowledge
- Rules/algorithms
- Goals

Example: If the camera detects a red traffic light, the agent program decides that the car should stop.

> **Definition:** An agent program maps the percept sequence received by an agent to an appropriate action.

---

3. Actuators

Actuators are used by the agent to **perform actions on the environment**.
Examples:

- Motors
- Wheels
- Robotic arms
- Speakers
- Brakes

Example: After deciding to stop, the self-driving car activates its brakes through actuators.

Definition: Actuators are mechanisms through which an agent performs actions on its environment.

Agent = Architecture + Program

An intelligent agent is commonly described as:
Agent = Architecture + Agent Program

Architecture

The **architecture** is the physical or computational platform on which the agent operates.
Examples:

- Robot hardware
- Computer
- Smartphone
- Self-driving car

Agent Program

The **agent program** contains the logic that determines what the agent should do based on its percepts.

Example: Robot Vacuum Cleaner

Suppose a robot vacuum is cleaning a room.
Room / Environment
↓
Sensors detect dirt
↓
Agent Program
↓
Decision: Clean the area
↓
Motor / Actuator
↓
Robot moves and cleans

Here:

- **Environment** → Room
- **Sensors** → Dirt and obstacle sensors
- **Agent Program** → Decides where to move

- **Actuators** → Wheels and cleaning mechanism

9)Problem Solving in AI

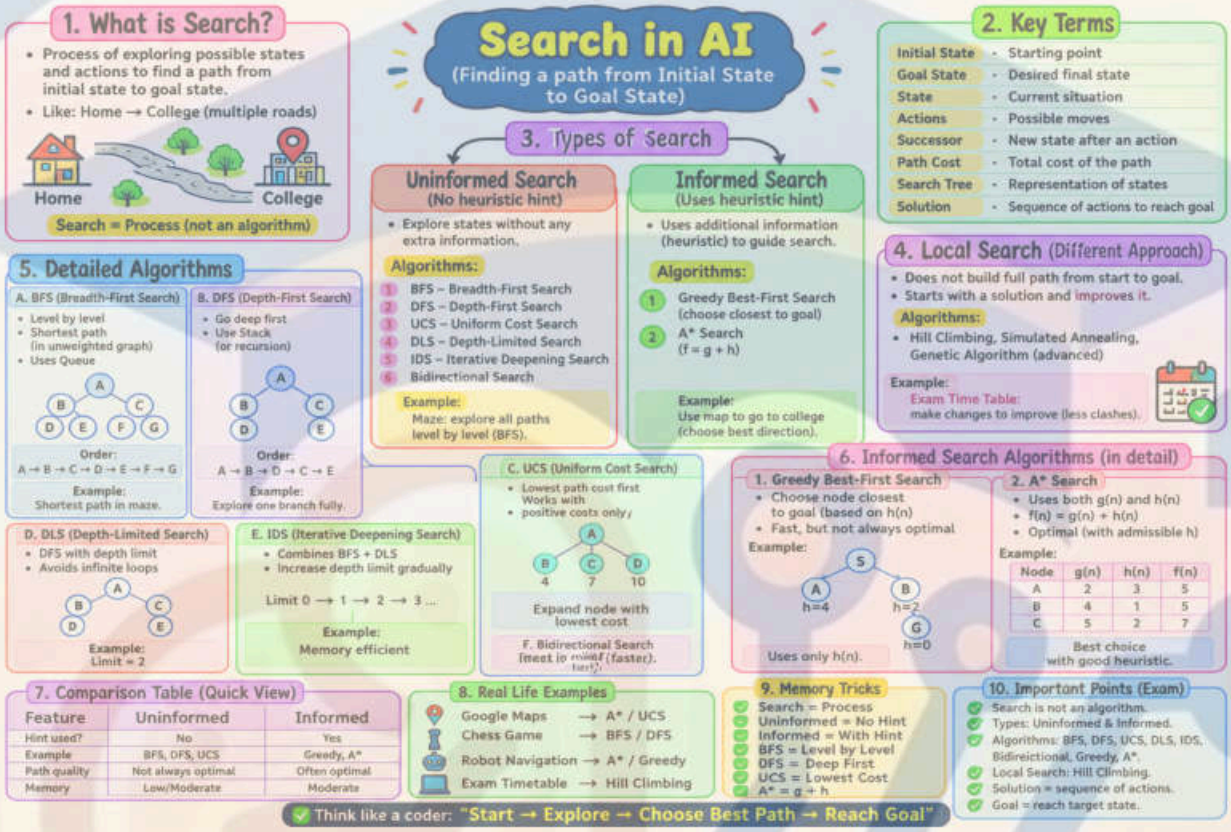
Problem Solving in AI is the process of finding a sequence of actions that takes an agent from an **initial state to a desired goal state**.
Example:
Initial State: Home
Goal State: College
Actions: Take different roads
Problem: Which route should you take?
Solution: A sequence of roads/actions that takes you from Home → College.

10)Solution Searching

Solution Searching is the process of exploring different possible states or paths to find a sequence of actions that reaches the goal.
The agent starts from the **initial state** and explores the **search space** until it finds a goal state.
Initial State
↓
Possible Actions
↓
New States
↓
More States
↓
Goal State

Example

If a person wants to travel from **A to D**:
A → B → D
A → C → D
A → B → C → D
The search algorithm examines these possibilities and finds a suitable path.



11)Uninformed Search Strategies

Uninformed search, also called **blind search**, searches for a solution **without using additional information about how close a state is to the goal**.
It only uses information provided by the problem definition.

Common Uninformed Search Algorithms

- **BFS — Breadth-First Search**
- **DFS — Depth-First Search**
- Uniform Cost Search
- Depth-Limited Search
- Iterative Deepening Search

12)Informed (Heuristic) Search Strategies

Informed search uses additional knowledge called a **heuristic** to guide the search toward the goal.
Instead of exploring blindly, the algorithm tries to choose the **most promising state**.

Common Informed Search Algorithms

- **Greedy Best-First Search**
- **A* Search**

Difference Between Uninformed and Informed Search

Feature	Uninformed Search	Informed Search
Meaning	Searches without extra knowledge about the goal	Uses extra knowledge to guide the search
Heuristic	Does **not** use a heuristic function	**Uses** a heuristic function
Knowledge	Uses only problem information	Uses problem information + heuristic information
Direction	Searches more blindly	Searches toward promising states
Efficiency	Usually explores more states	Usually explores fewer states
Examples	BFS, DFS, Uniform Cost Search	A*, Greedy Best-First Search
Function	No `h(n)` required	Uses `h(n)`
Complexity	Can be higher for large search spaces	Can be lower if heuristic is effective

13)Heuristic Function

A **Heuristic Function** is a function used in **Informed (Heuristic) Search** to estimate the cost or distance from a current state to the **goal state**. It helps the search algorithm decide which state is more promising and should be explored first.
It is generally represented as:
h(n)\boxed{h(n)}
where **h(n)** = estimated cost from node/state **n** to the goal.

How it works

Suppose an AI agent wants to travel from **Delhi to Mumbai**. At a particular city, the heuristic function can estimate the remaining distance to Mumbai.
For example:
Delhi → Jaipur → Ahmedabad → Mumbai
If:

- h(Jaipur) = 900 km
- h(Ahmedabad) = 500 km

then the algorithm may consider Ahmedabad more promising because its estimated distance to the goal is smaller.

Important Points

1. **Used in informed search**
   Heuristic functions are mainly used by algorithms such as **Greedy Best-First Search** and **A***.
2. **Estimates remaining cost**
   It does not normally give the exact cost; it provides an estimate.
3. **Helps choose the next state**
   A lower heuristic value generally indicates that the state appears closer to the goal.
4. **Can make search faster**
   A good heuristic can reduce the number of states that the algorithm needs to explore.
5. **Depends on the problem**
   Different problems require different heuristic functions.

Example: 8-Puzzle

In the **8-puzzle**, one possible heuristic is the **number of misplaced tiles**.
Current State:
1 2 3
4 5 6
8 7 _

Goal State:
1 2 3
4 5 6
7 8 _
Here, tiles **7 and 8** are misplaced.
Therefore:
$h(n)=2h(n) = 2$
So the heuristic estimates that the current state is **2 moves/placements away** from the goal, depending on the chosen heuristic definition.

Heuristic in A* Search

A* uses both the actual cost and the estimated cost:
$f(n)=g(n)+h(n)\boxed{f(n)=g(n)+h(n)}$
Where:

- **g(n)** = actual cost from the initial state to node n
- **h(n)** = estimated cost from node n to the goal
- **f(n)** = estimated total cost of the solution through n

Local Search Algorithms and Optimization Problems

These two topics are closely related. **Local Search** is a method of searching for a good solution by examining nearby solutions, while an **Optimization Problem** focuses on finding the best possible solution according to some objective.

---

1. Local Search Algorithms

> **Local Search Algorithms are search techniques that start with an initial solution and repeatedly examine neighboring states to find a better solution. They focus mainly on the current state and its nearby states rather than maintaining a complete search path.**

Unlike BFS or DFS, local search algorithms generally do **not need to store the entire search tree**. This makes them useful for problems where the goal is to find an **optimal or near-optimal solution**.

Basic Working

Initial State
↓
Check Neighboring States
↓
Choose a Better State
↓
Check Its Neighbors
↓
Continue Improving
↓
Good / Optimal Solution

For example, suppose we want to find the highest point on a hill:



The algorithm looks at nearby positions and moves toward a position with a better value.

Main Characteristics

1. **Starts with an initial state**
   The algorithm begins with some possible solution.
2. **Examines neighboring states**
   It looks at solutions that are slightly different from the current solution.
3. **Moves toward a better state**
   If a neighboring state is better, the algorithm moves there.
4. **Uses less memory**
   It usually stores only the current state and some neighboring states.
5. **May find a local optimum**
   It can sometimes stop at a solution that is better than its neighbors but is not the best solution overall.

Common Local Search Algorithms

- **Hill Climbing**
- **Simulated Annealing**
- **Local Beam Search**
- **Genetic Algorithms**

---

2. Optimization Problems

Definition

> **An optimization problem is a problem in which the objective is to find the best possible solution from a set of feasible solutions according to a given objective or cost function.**

The "best" solution can mean:

- **Minimum cost**
- **Minimum distance**
- **Minimum time**
- **Maximum profit**
- **Maximum performance**

- **Maximum accuracy**

Example

Suppose a delivery company wants to find the shortest route for delivering packages.
Route A → 50 km
Route B → 40 km
Route C → 35 km
Route D → 60 km
The optimization objective is to find the route with **minimum distance**.
So:
Best solution=Route C\text{Best solution} = \text{Route C}
Here, **distance** is the objective/cost that we want to minimize.

Local Search for Optimization

Local search is particularly useful for optimization problems because it can continuously improve a candidate solution.
For example:
Solution A → Cost = 100
↓
Solution B → Cost = 80
↓
Solution C → Cost = 60
↓
Solution D → Cost = 45
The algorithm keeps moving toward solutions with better objective values.
However, it may reach a **local optimum** instead of the **global optimum**.

Local Optimum vs Global Optimum

Local optimum:
A solution that is better than all of its nearby solutions but may not be the best solution overall.
Global optimum:
The best solution among all possible solutions.



Local Maximum Global Maximum

Problems in Local Search

Local search can face several difficulties:

1. Local Maximum

The algorithm reaches a state that is better than its neighbors but is not the global best.

2. Plateau

Many neighboring states have the same value, so the algorithm cannot determine which direction is better.

3. Ridge

The best direction may require a sequence of moves that the algorithm cannot easily discover by simply choosing the immediate best neighbor.

Difference Between Local Search and Traditional Search

Feature	Local Search	Traditional Search
Main goal	Find a good/optimal solution	Find a path to the goal
Memory	Usually low	Can require high memory
Focus	Current state and neighbors	Search tree/graph
Path tracking	Usually not important	Usually important
Common examples	Hill Climbing, Simulated Annealing	BFS, DFS, A*

Local Search in Continuous Spaces

Definition

Local Search in Continuous Spaces is a search technique used to find an optimal or near-optimal solution when the possible states or variables can take any value within a continuous range.

In a **continuous space**, variables are not limited to fixed, separate values. They can take decimal or real-number values.

Example

Consider a variable xx:

Discrete space:

x = 1, 2, 3, 4, 5

Continuous space:

x = 1.1, 1.25, 1.37, 2.456, 3.891, ...

There can be infinitely many possible values between two points.

How Local Search Works in Continuous Spaces

The algorithm starts with an initial solution and gradually changes its values to find a better solution.

Initial Solution
↓
Check Nearby Values
↓
Move toward Better Value
↓
Check Again
↓
Continue Improving
↓
Optimal / Near-Optimal Solution

For example, suppose we want to **maximize**:

f(x)=−(x−5)2+25f(x) = -(x-5)^2 + 25

The maximum value occurs at:

x=5x = 5

A local search algorithm may start at x=2x=2, then try nearby values:

x = 2.0
↓
x = 3.0
↓
x = 4.0
↓
x = 4.5

↓
x = 5.0 ← Maximum
The algorithm gradually moves toward the better solution.

Important Characteristics

1. Continuous Variables

The states are represented using real-valued variables such as:
x=2.35,y=7.82x = 2.35,\quad y = 7.82
rather than only integer or discrete values.

2. Neighbourhood

Instead of checking a fixed list of neighboring states, the algorithm considers small changes in the current values.
For example:
x⟶x+Δxx \rightarrow x+\Delta x
where Δx\Delta x is a small change.

3. Objective Function

A function is used to measure how good a solution is.
For example:
f(x,y)f(x,y)
The algorithm tries to **maximize** or **minimize** this function.

4. Gradient-Based Methods

A common approach is to use the **gradient**, which indicates the direction in which the function increases or decreases.
For maximization:
xnew=xold+α∇f(x)x_{new}=x_{old}+\alpha \nabla f(x)
where:

- α\alpha = step size/learning rate
- ∇f(x)\nabla f(x) = gradient
- xx = current solution

This idea is used in techniques such as **gradient ascent/descent**.

5. Local Optimum

The algorithm may reach a point where nearby solutions are not better, even though a better solution may exist elsewhere. This is called a **local optimum**.

Example in AI/ML

Suppose a machine-learning model has parameters w1,w2,w3w_1,w_2,w_3, and we want to minimize its error:
Error=f(w1,w2,w3)Error = f(w_1,w_2,w_3)
The weights can have continuous values such as:
$w_1 = 0.52$
$w_2 = -1.37$
$w_3 = 2.84$
A local-search optimization method can continuously adjust these values to reduce the error.

Applications

Local search in continuous spaces is commonly used in:

- Machine Learning model optimization
- Neural network training
- Robotics
- Engineering design
- Function optimization
- Parameter tuning
- Resource allocation

Exam-ready Definition

Local search in continuous spaces is a search and optimization technique used when the possible states or variables can take continuous real-valued values. It starts from an initial solution and makes small changes to the variables, moving toward solutions with better objective values. Gradient-based methods are commonly used to efficiently find an optimal or near-optimal solution.

1. ***Unit 2:***

1. Logical Agents

A logical agent is an intelligent agent that uses logic and a knowledge base to represent information about the world and perform reasoning to decide what action to take.

Environment
↓
Percepts
↓
Knowledge Base
↓
Reasoning / Inference
↓
Decision
↓
Action

Example:

A robot knows:

- It is raining.
- If it is raining, the ground is wet.

Using logic, it can infer:

Ground is wet.

Characteristics of Logical Agents

1. **Knowledge-based** – stores information in a knowledge base.
2. **Reasoning ability** – derives new information using logic.
3. **Goal-oriented** – uses knowledge to achieve goals.
4. **Perception** – receives information from the environment.
5. **Decision-making** – selects actions based on reasoning.
6. **Action** – interacts with the environment through actuators.

Advantages

- Can make decisions using stored knowledge.
- Can derive new information from existing facts.
- Useful for reasoning and problem-solving.
- Can represent complex relationships using logic.
- Useful in expert systems and knowledge-based AI.

Limitations

- Requires accurate and well-defined knowledge.
- Complex reasoning can require significant computation.

- Handling uncertain or incomplete information can be difficult using basic logic alone.

2. Propositional Logic

Propositional Logic is a formal system in which knowledge is represented using propositions that are either True or False and are combined using logical operators.

Example:
P = It is raining.
Q = The ground is wet.
We can write:
P⟶QP \rightarrow Q
Meaning:
If it is raining, then the ground is wet.

Common operators

Operator	Meaning
¬P	NOT
P ∧ Q	AND
P ∨ Q	OR
P ⟶ Q	IMPLIES
P ↔ Q	IF AND ONLY IF

---

3. Inference

Inference is the process of deriving new knowledge or conclusions from existing facts and rules using logical reasoning.

Example:
Fact: Human(Sonam)
Rule: Human(x) ⟶ Mortal(x)
Conclusion: Mortal(Sonam)
The system uses the fact and rule to derive a new fact.
So:
Facts + Rules
↓
Inference

↓
New Knowledge

---

4. First-Order Logic (FOL)

> **First-Order Logic is a logical system that represents objects, their properties, and relationships between objects using predicates, variables, constants, functions, and quantifiers.**

FOL is more powerful than propositional logic because it can represent **objects and relationships**.
Example:
Human(Socrates)
means Socrates is human.
Rule:
Human(x)⟶Mortal(x)Human(x) \rightarrow Mortal(x)
Therefore:
Mortal(Socrates)Mortal(Socrates)

Important components

- **Constants:** `Socrates`, `Ram`, `Delhi`
- **Variables:** `x`, `y`
- **Predicates:** `Human(x)`, `Student(x)`
- **Quantifiers:** `∀` and `∃`

---

5. Inference in First-Order Logic

> **Inference in First-Order Logic is the process of deriving new facts or conclusions from existing FOL facts and rules.**

Example:
Given:
Student(Ram)Student(Ram)
and:
Student(x)⟶Studies(x)Student(x) \rightarrow Studies(x)
We can infer:
Studies(Ram)Studies(Ram)
FOL inference commonly involves techniques such as:

- Unification
- Forward chaining
- Backward chaining
- Resolution

6. Forward Chaining

Forward Chaining is a data-driven inference method that starts with known facts and repeatedly applies rules to derive new facts until the required conclusion is obtained or no new facts can be generated.

Steps

1. Start with the available facts.
2. Check the rules whose conditions match those facts.
3. Apply the matching rules.
4. Add the newly derived facts to the knowledge base.
5. Continue until the goal is derived or no more rules can be applied

Example:
Fact:
Human(Ram)

Rule:
Human(x) ⟶ Mortal(x)
Forward chaining:
Human(Ram)
↓
Human(x) ⟶ Mortal(x)
↓
Mortal(Ram)
It moves:
Facts ⟶ Rules ⟶ New Facts ⟶ Conclusion

---

7. Backward Chaining

Backward Chaining is a goal-driven inference method that starts with a desired goal and works backward to determine which facts and rules are needed to prove that goal.

Example:
Goal:
Mortal(Ram)
Rule:
Human(x) ⟶ Mortal(x)
The system asks:

To prove `Mortal(Ram)`, do I need `Human(Ram)`?

If:
Human(Ram)
is already a fact, the goal is proved.

Flow:
Goal
↓
Find Rule
↓
Required Facts
↓
Check Facts
↓
Goal Proved

Forward vs Backward Chaining

Feature	Forward Chaining	Backward Chaining
Approach	Data-driven	Goal-driven
Starts with	Known facts	Goal
Direction	Forward	Backward
Process	Facts → Conclusion	Goal → Required facts
Main purpose	Derive new facts	Prove a specific goal
Example	Expert systems	Prolog

8. Unification

Unification is the process of finding substitutions for variables that make two logical expressions identical or compatible.

Example:
Student(x)
Student(Ram)
We can substitute:
x=Ramx = Ram
After substitution:
Student(Ram)

Student(Ram)
Both expressions match.
Another example:
Likes(x, Mango)
Likes(Ravi, Mango)
Therefore:
x=Ravix = Ravi
This process is called **unification**.

---

9. Resolution

> **Resolution is an inference rule used in logic to derive a new conclusion by combining two clauses that contain complementary literals.**

Example:
P ∨ Q
¬Q ∨ R
Since Q and ¬Q are complementary, they can be resolved to produce:
P ∨ R

Difference Between Unification and Resolution

Unification	Resolution
Makes two expressions match	Derives a new logical conclusion
Uses substitutions	Uses complementary literals
Works mainly with variables/terms	Works with logical clauses
Example: `Student(x)` + `Student(Ram)`	`P ∨ Q` + `¬Q ∨ R`
Produces a substitution	Produces a new clause
Used by resolution	Can use unification in FOL

Knowledge Representation

Knowledge Representation (KR) is a fundamental concept in Artificial Intelligence used to represent knowledge about the real world in a form that a computer can understand, reason with, and use for decision-making.
In Unit 2, the important topics are:

1. **Ontological Engineering**
2. **Categories and Objects**
3. **Events**

---

1. Ontological Engineering

> **Ontological Engineering is the process of designing and developing an ontology that formally represents concepts, categories, properties, relationships, and rules of a particular domain.**

An **ontology** describes **what things exist in a particular domain and how those things are related to each other**.

Simple Example

Suppose we want to create an ontology for a **University**.
We can represent:

- University
  - Department
  - Teacher
  - Student
  - Course

Relationships:

- Student **enrolls in** Course
- Teacher **teaches** Course
- Student **belongs to** Department
- Department **is part of** University

So, the ontology gives AI a structured understanding of the university domain.

Main Components

1. Categories/Classes
Represent groups of similar objects.
Example:

- Student
- Teacher
- Course
- Department

2. Objects/Instances
Specific members of a category.

Example:

- Sonu → Student
- AI → Course

3. Properties

Describe characteristics of objects.

Example:

- Student has `name`
- Student has `roll_no`
- Course has `credits`

4. Relationships

Describe how objects are connected.

Example:

- Student → enrolls → Course
- Teacher → teaches → Course

5. Rules/Constraints

Specify restrictions or logical conditions.

Example:

Every student must be enrolled in at least one course.

Purpose of Ontological Engineering

- Organize knowledge systematically
- Represent concepts and relationships
- Allow machines to understand a domain
- Support logical reasoning
- Enable knowledge sharing and reuse
- Reduce ambiguity in knowledge representation

Exam-ready Definition

Ontological Engineering is the process of creating and maintaining an ontology that formally represents the concepts, categories, objects, properties, relationships, and constraints of a specific domain. It helps AI systems organize knowledge and perform reasoning effectively.

---

2. Categories and Objects

This topic deals with **classifying real-world things into categories and representing individual things as objects**.

Category

A **category** is a group or class containing objects that share common properties or characteristics.
Examples:

- **Animal** → Dog, Cat, Horse
- **Vehicle** → Car, Bus, Bike
- **Student** → individual students

Object

An **object** is a particular entity or instance belonging to a category.
Example:

- Category: **Student**
- Objects: **Sonu, Rahul, Amit**

Here, Sonu is an object/instance of the Student category.

Example

Consider:
Category: Car
Objects:

- Car1 = Honda City
- Car2 = Toyota
- Car3 = Hyundai

Properties:

- color
- model
- speed
- manufacturer

So:

Car → Category
Honda City → Object/Instance

Hierarchy of Categories

Categories can be arranged in a hierarchy.
For example:



Sedan SUV

This hierarchy allows AI to inherit common properties.

For example:

Vehicle

↓

Car

↓

Sedan

If **Vehicle** has the property `has wheels`, then Car and Sedan can inherit this property.

Why Categories and Objects Are Important

They help AI:

- Organize knowledge
- Classify entities
- Represent real-world objects
- Share common properties
- Support inheritance
- Perform reasoning

Exam-ready Definition

> **Categories are groups of objects that share common properties or characteristics, while objects are individual entities or instances belonging to those categories. Categories help AI organize knowledge and support inheritance and reasoning.**

---

3. Events

Definition

> **An event is an occurrence or action that happens at a particular time or during a particular period and may involve one or more objects.**

In AI, events are important because they allow a system to represent **actions, activities, changes, and occurrences in the real world**.

Examples

- Student **attends** a class.
- Teacher **teaches** a lecture.
- Person **buys** a product.
- Car **moves** from one location to another.

- Student **submits** an assignment.

These are all events because something happens or changes.

Components of an Event

An event can be described using:

1. Participants

Objects involved in the event.

Example:

Sonu buys a book.

Participants:

- Sonu
- Book

2. Action

What happens.

Buys

3. Time

When the event happens.

10:00 AM

4. Location

Where it happens.

Book Store

Example

Event:

Sonu buys a laptop.

We can represent:

Event: Buying

|

|-- Buyer: Sonu

|-- Object: Laptop

|-- Time: 5 PM

|-- Location: Shop

This allows an AI system to understand not only **what objects exist**, but also **what happened to those objects**.

Events and Change

Events can cause changes in the state of objects.

Example:

Before Event:

Money = ₹50,000

Laptop = Shop

↓

Buying Event

↓

After Event:
Money = ₹30,000
Laptop = Sonu
Therefore, events can be used to represent **state changes**.

Importance of Events in AI

Events are useful for:

- Understanding real-world activities
- Representing actions
- Tracking changes over time
- Planning and decision-making
- Natural Language Processing
- Robotics
- Expert systems
- Reasoning about time and actions

Exam-ready Definition

> **Events are occurrences or actions that take place in the real world and involve one or more objects. They can be represented using participants, actions, time, location, and effects, and are used in AI to represent activities and changes in the state of objects.**

Quick Revision

Topic	Meaning	Example
Ontological Engineering	Creating a formal representation of knowledge in a domain	University ontology
Category	Group/class of similar objects	Student, Vehicle
Object	Individual instance of a category	Sonu, Honda City
Event	An action or occurrence involving objects	Sonu buys a laptop

Practical 1: Breadth First Search (BFS)

Aim

To write a Python program to implement **Uninformed Search using Breadth First Search (BFS)**.

Theory

Breadth First Search (BFS) is an uninformed search algorithm that explores a graph **level by level**. It first visits all nodes at the current depth before moving to the next depth.
BFS uses a **Queue (FIFO – First In, First Out)**.
Example:



BFS traversal:

A → B → C → D → E → F

Algorithm

1. Start from the initial node.
2. Add the starting node to a queue.
3. Mark it as visited.
4. Remove the first node from the queue.
5. Visit its unvisited neighboring nodes and add them to the queue.
6. Repeat until the queue becomes empty or the goal is found.

Python Program

Python

```python
from collections import deque

def bfs(graph, start, goal):
    queue = deque([start])
    visited = set()

    while queue:
        node = queue.popleft()

        if node not in visited:
            print(node, end=" ")
            visited.add(node)

            if node == goal:
                print("\nGoal found!")
                return
```

```
            for neighbor in graph[node]:
                if neighbor not in visited:
                    queue.append(neighbor)

    print("\nGoal not found!")


# Graph
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

# BFS
bfs(graph, 'A', 'F')
```

Output

A B C D E F
Goal found!

Explanation

- `deque` is used to implement the **queue**.
- `queue.popleft()` removes the first element.
- `visited` keeps track of already visited nodes.
- BFS explores nodes level by level.

Complexity

- **Time:** `O(V + E)`

- **Space:** $O(V)$

where V = number of vertices and E = number of edges.

Conclusion

Thus, the **Breadth First Search algorithm** was successfully implemented using Python.

Viva Questions

Q1. What type of search is BFS?
BFS is an **uninformed search algorithm**.
Q2. Which data structure does BFS use?
BFS uses a **Queue**.
Q3. What does FIFO mean?
First In, First Out.

Practical 2: A* Search

Aim

To write a Python program to implement **Informed Search using the A* algorithm**.

Theory

A* is an informed search algorithm that uses both the actual cost of reaching a node and an estimated cost from that node to the goal.
It uses the evaluation function:
$f(n)=g(n)+h(n)f(n) = g(n) + h(n)$
Where:

- $g(n)$ = actual cost from start node to node n
- $h(n)$ = estimated cost from node n to goal
- $f(n)$ = estimated total cost of the path through node n

Example Graph



Suppose:
Start = A

Goal = D
Heuristic values:
h(A) = 3
h(B) = 2
h(C) = 1
h(D) = 0

Python Program

Python

```python
import heapq

def a_star(graph, heuristic, start, goal):

    priority_queue = []

    # (f, g, node, path)
    heapq.heappush(priority_queue, (heuristic[start], 0, start, [start]))

    visited = set()

    while priority_queue:

        f, g, current, path = heapq.heappop(priority_queue)

        if current in visited:
```

```
if current in visited:
    continue

visited.add(current)

if current == goal:
    print("Path:", " -> ".join(path))
    print("Total Cost:", g)
    return

for neighbor, cost in graph[current]:

    if neighbor not in visited:

        new_g = g + cost
        new_f = new_g + heuristic[neighbor]

        heapq.heappush(
            priority_queue,
            (new_f, new_g, neighbor, path +      ghbor])
        )
```

```
    print("Goal not found!")


# Graph
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('A', 1), ('D', 2)],
    'C': [('A', 4), ('D', 1)],
    'D': [('B', 2), ('C', 1)]
}

# Heuristic values
heuristic = {
    'A': 3,
    'B': 2,
    'C': 1,
    'D': 0
}

# A* Search
a_star(graph, heuristic, 'A', 'D')
```

Practical 3: Truth Table of Propositional Logic

Aim

To write a Python program to compute the **truth table of a statement in propositional logic**.

Theory

Propositional Logic is a formal system in which statements are represented as propositions that can have one of two truth values:
True (T)
False (F)
Logical operators include:

Operator	Meaning
AND (∧)	True when both are true
OR (∨)	True when at least one is true
NOT (¬)	Reverses the truth value
IMPLIES (→)	False only when P is True and Q is False

We will generate the truth table for:
P∧QP \land Q

Truth Table

P	Q	P ∧ Q
T	T	T
T	F	F
F	T	F
F	F	F

Python Program

Python

```python
print("Truth Table for P AND Q")
print("-------------------------")
print("P\tQ\tP AND Q")

values = [True, False]

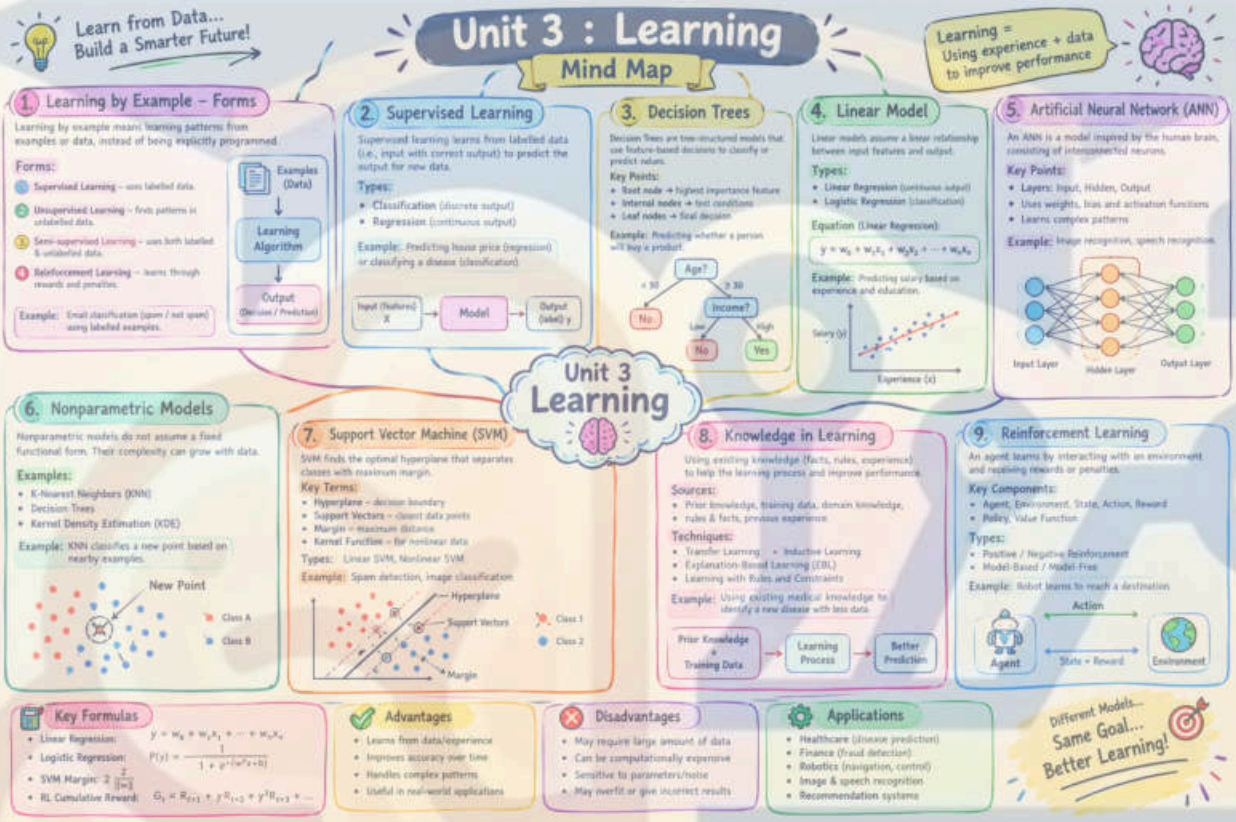
for P in values:
    for Q in values:
        result = P and Q

        print(
            str(P)[0],
            "\t",
            str(Q)[0],
            "\t",
            str(result)[0]
        )
```

Output

```
Truth Table for P AND Q
-------------------------
P	Q	P AND Q
T	T	T
T	F	F
F	T	F
F	F	F
```

UNIT 3 — LEARNING



1. Learning by Example — Forms of Learning

Definition

Learning by Example is a process in which an AI system learns patterns, relationships, or rules from examples and uses them to make predictions or decisions about new data.
Example: An AI system is given images labelled "Cat" and "Dog." It learns patterns from these images and predicts whether a new image contains a cat or a dog.

Example

A spam filter ko agar thousands of emails diye jayein:

- "Win $1,000" → Spam
- "Congratulations! You won" → Spam
- "Meeting at 5 PM" → Not Spam

3. Forms of Learning

In AI learning is broadly classified in different forms

Important Forms

1. **Supervised Learning**
2. **Unsupervised Learning**
3. **Reinforcement Learning**

Other learning approaches include semi-supervised learning, which uses labelled and unlabelled data, and self-supervised learning, which creates learning targets from the data itself.

4. Supervised Learning ⭐⭐⭐

Supervised Learning is a type of machine learning in which a model learns from **labeled training data**.
In this both input and correct output are given
It is similar to how a teacher teach student using solved example

Working

1. Collect labelled training data.
2. Select a suitable learning algorithm.
3. Train the model using input-output examples.
4. Compare predictions with actual outputs.
5. Evaluate the trained model on unseen data.

Types of Supervised Learning

A. Classification: Predicts a category or class.
Examples: Spam or Not Spam, Pass or Fail, Cat or Dog.
B. Regression: Predicts a numerical value.
Examples: House prices, temperature, and sales forecasting.

Advantages

- Learns from known examples.
- Can make predictions on new data.
- Useful for classification and numerical prediction.

Limitations

- Requires labelled training data.
- Poor-quality data may produce inaccurate predictions.
- Models may overfit the training data.

5. Decision Trees ⭐ ⭐ ⭐

Definition

A **Decision Tree** is a supervised learning algorithm that represents decisions in the form of a **tree structure**.
Isme:

- **Root** → starting decision
- **Branches** → possible outcomes
- **Internal nodes** → conditions/tests
- **Leaf nodes** → final answer/class

Example

Suppose we want to decide whether to play cricket:



Components

- **Root Node: The starting point of the tree.**
- **Internal Node: Represents a test on a feature.**
- **Branch: Represents the outcome of a test.**
- **Leaf Node: Represents the final prediction.**

Working

1. **The algorithm examines the training data.**
2. **It selects a feature for splitting the data.**
3. **The data is divided into smaller groups.**
4. **The process repeats until a stopping condition is met.**
5. **The final tree predicts the output for new inputs.**

Advantages

- Easy to understand
- Easy to visualize
- Requires relatively little data preparation
- Useful for classification and regression

Disadvantages

- Can overfit
- Large trees can become complicated
- Small changes in data can sometimes change the tree

Limitations

- Deep trees may overfit.
- Small changes in data can produce different trees.
- A single tree may perform poorly on complex patterns.

Linear Model

A **Linear Model** assumes that the relationship between input variables and output can be represented using a linear equation.
A Linear Model is a mathematical model in Machine Learning that represents the relationship between input variables (features) and an output using a linear equation.
It learns the relationship from training data and uses that relationship to predict the output for new inputs.
In simple words: A Linear Model tries to predict an output by finding a straight-line relationship between the input and output.
Example: Predicting a student's marks based on the number of hours studied.

- Input (x): Hours studied
- Output (y): Marks obtained
- Model: Learns how marks change as study hours increase.

2. Basic Linear Model Equation

For one input variable, the equation is:

$$y = mx + c$$

Where:

- y= predicted output
- x = input variable
- m = slope or coefficient (how much y changes when x increases by one unit)
- ccc = intercept (the predicted value of y when x=0)

7. Applications of Linear Models

1. Education: Predicting student marks.
2. Business: Forecasting sales and revenue.
3. Real estate: Estimating property prices.

8. Advantages

- Simple to understand and implement.
- Fast to train on many datasets.
- Easy to interpret when the features are meaningful.

9. Disadvantages

- Cannot naturally capture complex nonlinear relationships without additional features or transformations.
- Can be affected by outliers.
- Highly correlated input features can make coefficients difficult to interpret.
- Performance depends on data quality and whether the model's assumptions fit the problem.
- May underfit when the actual relationship is complex.

Decision Tree Algorithms

- ID3
- C4.5
- CART (Classification and Regression Trees)

10. Linear Model vs. Artificial Neural Network

Basis	Linear Model	Artificial Neural Network
Structure	Weighted sum of inputs plus bias	Layers of interconnected neurons
Complexity	Usually simpler	Can be highly complex

Basis	Linear Model	Artificial Neural Network
Relationship	Linear in its parameters	Can learn nonlinear relationships using nonlinear activations
Training	Often computationally efficient	Can require more data and computing
Interpretability	Generally easier	Often harder
Example	Basic price prediction	Image recognition

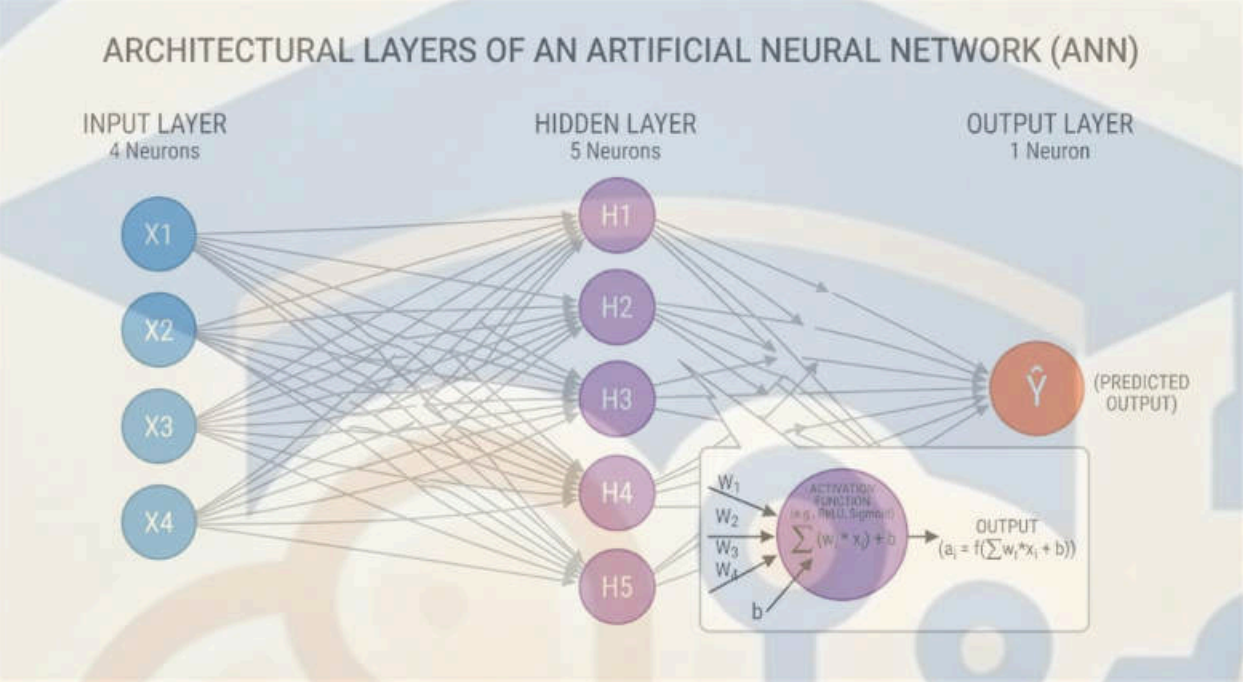
Artificial Neural Network (ANN) — 10 Marks Answer

1. Definition

An Artificial Neural Network (ANN) is a machine learning model inspired by the structure and functioning of the human brain. It consists of interconnected processing units called artificial neurons, which work together to learn patterns from data and make predictions or decisions. ANN is used for tasks such as image recognition, speech recognition, classification, regression, and pattern detection.
Example: An ANN can learn from images of cats and dogs and then predict whether a new image contains a cat or a dog.

2. Architecture of an Artificial Neural Network

A basic ANN consists of three types of layers:



A. Input Layer

- Receives input data from the outside world.
- Each input neuron represents a feature.
- Example: Pixel values of an image or a student's study hours.

B. Hidden Layer

- Processes information received from the input layer.
- Uses weights, biases, and activation functions.
- An ANN may contain one or multiple hidden layers.

C. Output Layer

- Produces the final prediction or result.
- Example: Cat/Dog classification or a predicted house price.

3. Main Components of an ANN

1. Artificial Neuron: A processing unit that receives inputs and calculates an output.
2. Weights: Numerical values that determine the influence of each input.
3. Bias: An additional parameter that helps adjust a neuron's output.
4. Activation Function: Determines the neuron's output and can introduce nonlinearity.
5. Loss Function: Measures the difference between predicted and actual outputs.
6. Optimizer: Updates the weights and biases to reduce the loss.

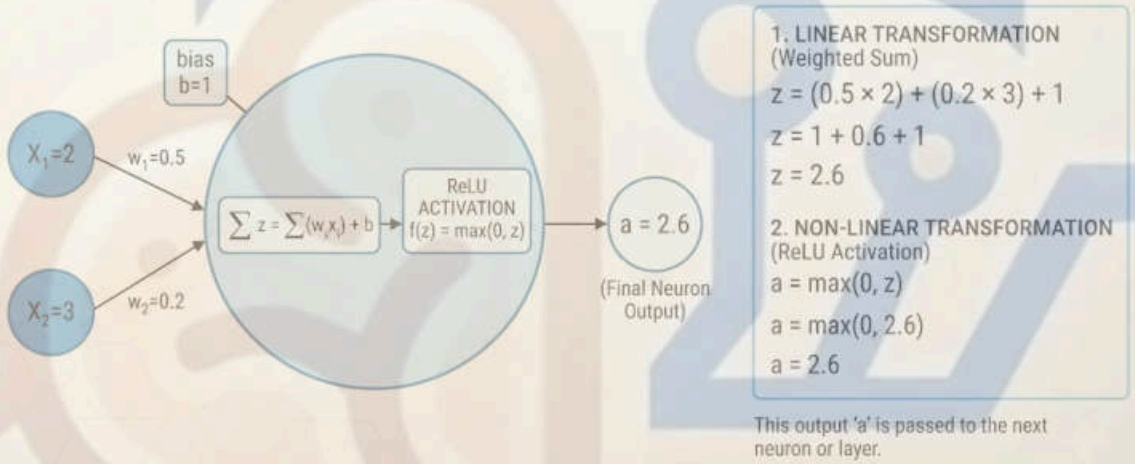
4. Working of an Artificial Neural Network

An ANN generally works through the following steps:

1. Input: The network receives input features.

2. Weighted Sum: Each neuron multiplies inputs by their weights and adds a bias.
3. Activation: An activation function processes the weighted sum.
4. Forward Propagation: Information moves through the network to generate a prediction.
5. Loss Calculation: The predicted output is compared with the correct output.
6. Backpropagation: The network calculates how the weights contributed to the error.
7. Weight Updating: An optimizer adjusts the weights and biases.
8. Iteration: The process repeats over training examples until the model learns useful patterns.

Single Artificial Neuron



Applications

- Image recognition
- Speech recognition
- NLP
- Face recognition
- Medical diagnosis
- Recommendation systems

Advantages

- Can learn complex patterns
- Works well with large datasets
- Useful for images, speech and text

Disadvantages

- Requires significant computational resources
- Can require lots of training data
- Often difficult to interpret ("black box")

6. Activation Functions

Activation functions help neural networks learn complex, nonlinear relationships.

Function	Formula / Purpose
Sigmoid	f(z)=11+e−zf(z)=\frac{1}{1+e^{-z}}f(z)=1+e−z1; outputs values between 0 and 1
ReLU	f(z)=max(0,z)f(z)=\max(0,z)f(z)=max(0,z); commonly used in hidden layers
Tanh	Outputs values between −1 and 1
Softmax	Converts output scores into a probability distribution across classes

7. Types of Artificial Neural Networks

1. Feedforward Neural Network: Information flows from input to output without cycles.
2. Convolutional Neural Network (CNN): Commonly used for image processing and computer vision.
3. Recurrent Neural Network (RNN): Designed to process sequential data.
4. Deep Neural Network (DNN): Contains multiple hidden layers.

Nonparametric Models

1. Definition

A Nonparametric Model is a machine learning model that does not assume a fixed functional form or a fixed number of parameters to represent the relationship between input and output. Its complexity can grow with the amount of training data.
These models learn patterns from data without requiring the relationship to follow a predefined equation such as a straight line.
Example: K-Nearest Neighbors (KNN) predicts a new data point's class by examining the classes of nearby data points.

2. Why Are Nonparametric Models Used?

A simple parametric model, such as Linear Regression, assumes a particular form for the relationship between input and output. However, real-world data may contain complex patterns.

Nonparametric models are useful when:

- The relationship between variables is unknown.
- The data follows a complex or nonlinear pattern.
- A fixed mathematical form may not fit the data well.

4. Types and Examples of Nonparametric Models

A. K-Nearest Neighbors (KNN)

KNN predicts the output for a new data point by finding its KKK nearest training examples.
Example: If 4 out of the 5 nearest data points belong to the Pass class, KNN may classify the new student as Pass.

B. Decision Trees

Decision Trees divide data using feature-based conditions and create a tree of decisions. Their structure can grow according to the training data.
Example: Predicting whether a customer will purchase a product based on age and income.

C. Kernel Methods

Some kernel-based methods, such as Support Vector Machines with nonlinear kernels, can model nonlinear relationships without explicitly constructing all the transformed features.
Example: Separating two classes when a straight decision boundary is insufficient.

D. Kernel Density Estimation (KDE)

KDE estimates the probability distribution of data using nearby observations rather than assuming a fixed distribution such as a normal distribution.
Example: Estimating the distribution of students' examination marks.

7. Advantages

- Flexible for complex data patterns.
- Can model nonlinear relationships.
- Makes fewer assumptions about the form of the data.
- Some methods are easy to understand and implement.
- Useful when the true relationship is unknown.

8. Disadvantages

- Some models require large amounts of memory.
- Prediction can be slow for large datasets.
- They can overfit when overly complex or poorly tuned.
- Performance may be sensitive to noise, irrelevant features, or parameter choices.
- Some models are difficult to interpret.

9. Applications

- Image and pattern recognition.
- Recommendation systems.
- Customer classification.

Support Vector Machine (SVM)

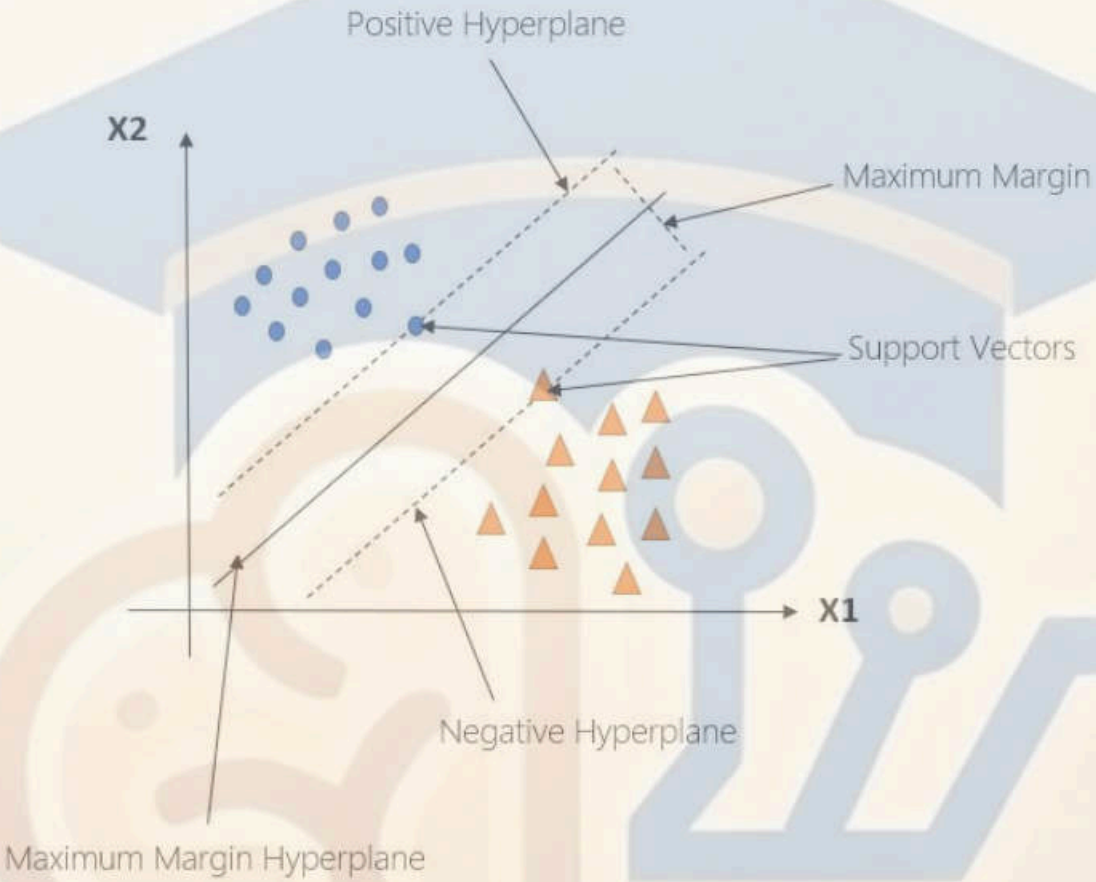
A Support Vector Machine (SVM) is a supervised machine learning algorithm used mainly for classification and also for regression. It finds an optimal decision boundary, called a hyperplane, that separates different classes of data while maximizing the margin between them.

SVM is particularly useful for high-dimensional data and can handle nonlinear classification using kernel functions.

Example: SVM can classify emails as Spam or Not Spam based on their features.

2. Basic Concept of SVM

SVM works by finding the best boundary that separates data points belonging to different classes.



3. Important Terms in SVM

1. Hyperplane: The decision boundary that separates classes. In two dimensions, it is a line; in three dimensions, it is a plane.

2. Support Vectors: The data points closest to the decision boundary. They play a key role in determining the position of the optimal hyperplane.

3. Margin: The distance between the decision boundary and the nearest data points from either class. SVM attempts to maximize this distance.

4. Kernel Function: A function that allows SVM to create nonlinear decision boundaries by implicitly representing data in a higher-dimensional feature space.

5. Classification Boundary: The boundary used to assign a new data point to a class.

4. Working of SVM

SVM generally works through the following steps:

1. Collect labelled data: Obtain training examples with their class labels.
2. Represent the data: Express each example using its features.
3. Find a separating hyperplane: Identify a boundary that separates the classes.
4. Identify support vectors: Find the closest training points to the boundary.
5. Maximize the margin: Choose the boundary with the largest feasible margin.
6. Use a kernel if required: For nonlinear data, use a suitable kernel function.
7. Classify new data: Determine which side of the decision boundary the new example lies on.

Advantages

- Effective in high-dimensional spaces
- Works well for classification
- Can handle complex boundaries using kernels

Disadvantages

- Can be computationally expensive for very large datasets
- Choosing the right kernel/parameters can be difficult

5. Mathematical Representation

For a linear SVM, the decision boundary is represented by:

$$w^T x + b = 0$$

Where:

- w = weight vector
- x = input feature vector
- b = bias
- $w^T x + b = 0$ = separating hyperplane

6. Types of SVM

A. Linear SVM

Used when classes can be separated reasonably well by a straight line or hyperplane.

Example: Separating two groups of data using a straight decision boundary.

B. Nonlinear SVM

Used when a straight boundary cannot adequately separate the classes. It uses kernel functions to model nonlinear boundaries.

Example: Separating data arranged in circular patterns.

7. Common Kernel Functions

Kernel	Purpose
Linear	Handles approximately linearly separable data
Polynomial	Models polynomial relationships
RBF (Radial Basis Function)	Handles complex nonlinear boundaries
Sigmoid	Provides a sigmoid-shaped kernel similarity function

The choice of kernel affects the decision boundary and model performance.

10. Applications of SVM

- Email filtering: Spam and non-spam classification.
- Healthcare: Disease classification.
- Computer vision: Image and object classification.
- Text classification: Categorizing documents.

Difference Between SVM and Decision Tree

Basis	SVM	Decision Tree
Main idea	Finds a separating hyperplane	Uses a sequence of decisions
Boundary	Maximum-margin boundary	Feature-based splits
Interpretability	Often less intuitive	Usually easy to interpret
Nonlinear patterns	Uses kernels when required	Uses multiple splits
Applications	Image and text classification	Classification and regression

Knowledge in Learning

1. Definition

Knowledge in Learning is the use of existing knowledge, facts, rules, relationships, and prior experience to help a machine learning system learn new information more efficiently and make better predictions or decisions.
Instead of learning everything from scratch, an AI system uses previously acquired knowledge to guide its learning process.
Example: A medical AI system that already knows the symptoms of common diseases can use this knowledge to learn to identify a new disease more efficiently.

2. Why Is Knowledge Important in Learning?

Knowledge helps a machine learning system learn efficiently, especially when training data is limited.
Its importance includes:

1. Faster Learning: Existing knowledge reduces the amount of learning required from scratch.
2. Better Accuracy: Relevant prior knowledge can improve predictions.
3. Less Training Data: Some tasks can be learned using fewer examples.

4. Better Generalization: Knowledge can help the system apply what it has learned to new situations.
5. Improved Decision-Making: Rules and facts can guide the system towards reasonable decisions.

3. Sources of Knowledge in Learning

Knowledge can come from several sources:

- Prior Knowledge: Information already available before training.
- Training Data: Examples from which patterns are learned.
- Domain Knowledge: Knowledge provided by experts in a particular field.
- Rules and Facts: Explicitly defined statements and logical relationships.
- Previous Experience: Information acquired from earlier tasks or interactions.

5. Techniques for Using Knowledge in Learning

A. Transfer Learning

Transfer Learning uses knowledge acquired from one task to improve learning on another related task.
Example: A model trained on millions of general images is adapted to identify plant diseases using a smaller collection of plant images.

B. Inductive Learning

Inductive Learning derives general rules or patterns from specific training examples.
Example: After observing several examples of sick plants with particular symptoms, a model learns patterns associated with plant disease.

C. Explanation-Based Learning (EBL)

Explanation-Based Learning uses existing domain knowledge to explain why a training example satisfies a concept and then generalizes that explanation into a rule.
Example: A system uses known physics rules to explain why an object falls and derives a general rule from the explanation.

D. Learning Using Rules and Constraints

An AI system can use predefined rules or constraints to guide its learning and avoid unreasonable predictions.
Example: A scheduling system uses rules stating that one classroom cannot host two classes at the same time.

6. Example of Knowledge in Learning

Suppose an AI system is learning to identify cats.
Without prior knowledge:

- It receives many labelled images.
- It must learn relevant visual patterns from the examples.

With prior knowledge:

- It may start with knowledge learned from a large image dataset.
- It can reuse visual features such as edges, shapes and textures.
- It can then adapt to identify cats using fewer task-specific examples.

This illustrates how prior knowledge can make learning more efficient.

7. Advantages

1. Reduces the time required to learn a new task.
2. Can reduce dependence on large labelled datasets.
3. Can improve accuracy when the existing knowledge is relevant.
4. Helps apply previous learning to related problems.
5. Can support more efficient and consistent decisions.

8. Disadvantages

1. Incorrect prior knowledge can lead to incorrect predictions.
2. Knowledge from an unrelated task may reduce performance.
3. Integrating expert knowledge into a model can be difficult.
4. Old knowledge may become outdated.
5. The system may become biased if the knowledge or rules contain bias.

9. Applications

- Healthcare: Using medical knowledge to assist disease diagnosis.
- Education: Adapting learning systems based on previously learned concepts.
- Robotics: Using physical rules to guide movement.

Reinforcement Learning (RL) — 10 Marks Answer

1. Definition

Reinforcement Learning (RL) is a type of Machine Learning in which an agent learns to make decisions by interacting with an environment and receiving rewards or penalties based on its actions.
The main objective of Reinforcement Learning is to learn a strategy that maximizes the total cumulative reward over time.
Example: A robot learns to navigate a room. It receives a positive reward for reaching its destination and a penalty for hitting an obstacle.
The main components are:

1. Agent: The learner or decision-maker, such as a robot.
2. Environment: The world in which the agent operates.
3. State (SSS): The current situation of the agent.
4. Action (AAA): A move or decision made by the agent.
5. Reward (RRR): Feedback indicating how desirable an outcome is.
6. Policy (π\piπ): The strategy the agent uses to select actions.
7. Value Function: Estimates the expected future cumulative reward from a state or state-action pair.

3. Working of Reinforcement Learning

Reinforcement Learning works through repeated interaction between the agent and the environment.

1. Observe the State: The agent observes the current situation.
2. Choose an Action: It selects an action according to its policy.
3. Perform the Action: The environment changes in response.
4. Receive Feedback: The agent receives a reward or penalty.
5. Update Knowledge: It updates its policy or value estimates based on experience.
6. Repeat: The process continues, helping the agent learn better decisions over time.

The goal is to maximize the expected cumulative reward, not merely the immediate reward.

5. Types of Reinforcement Learning

A. Positive Reinforcement

The agent receives a positive reward for desirable behaviour, encouraging it to repeat that behaviour.
Example: A robot receives a reward for reaching its destination safely.

B. Negative Reinforcement

In the broader behavioural-learning sense, negative reinforcement strengthens behaviour by removing or avoiding an undesirable condition. In RL implementations, negative reward signals or penalties are also commonly used to discourage undesirable actions, though these are not technically identical concepts.

Example: Designing a robot's reward function to discourage unsafe movement near obstacles.

C. Model-Based Reinforcement Learning

The agent uses or learns a model of how the environment changes, then uses that model to plan actions.

D. Model-Free Reinforcement Learning

The agent learns a policy or value estimates directly from experience without requiring an explicit model of the environment.
Examples include Q-learning and SARSA.

Applications

- Game playing
- Robotics
- Autonomous vehicles
- Recommendation systems
- Resource management

Advantages

- Does not require labeled training data
- Learns through experience
- Useful for sequential decision-making

Disadvantages

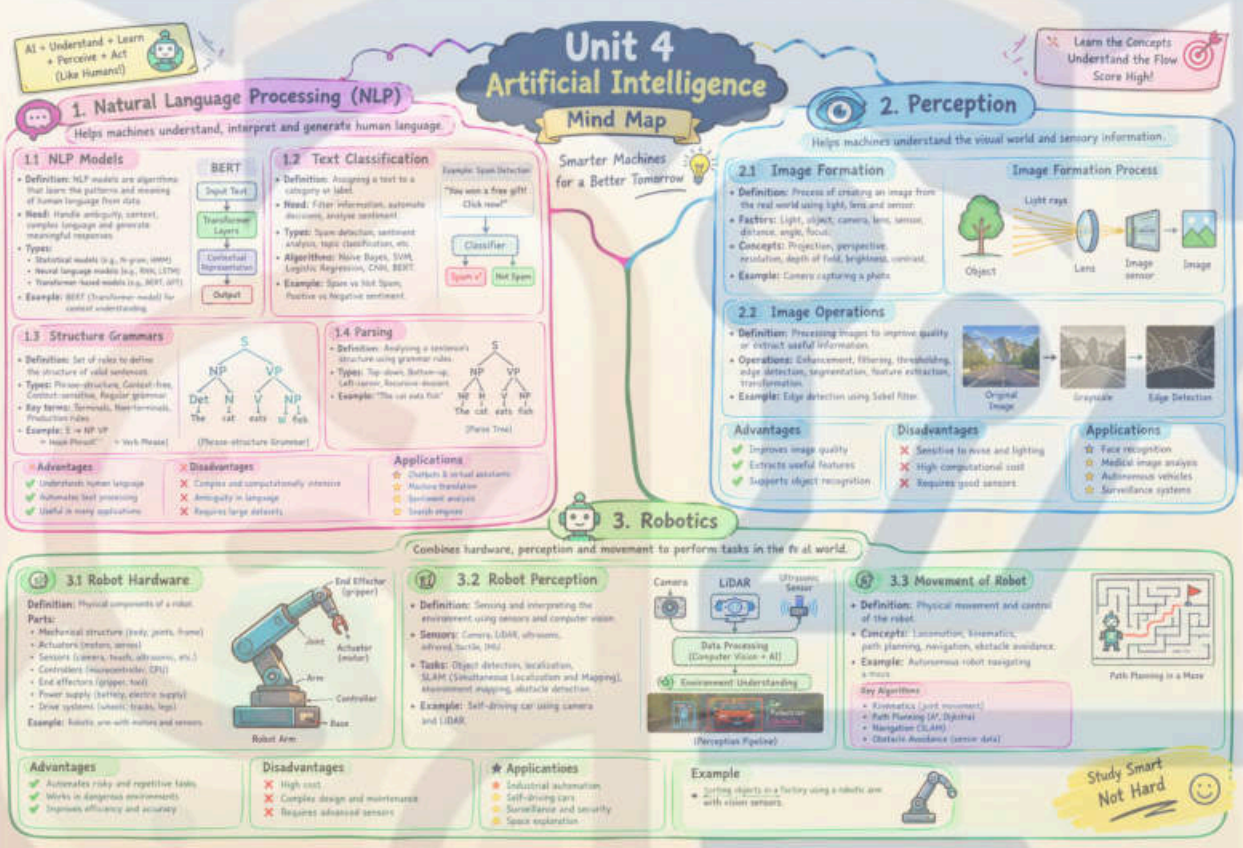
- Can require many trials
- Training can be slow
- Poor reward design can lead to unwanted behavior

🔥 Super Important Differences

Supervised vs Reinforcement Learning

Supervised	Reinforcement
Uses labeled data	Uses rewards/penalties
Correct answer is provided	Correct answer is not directly provided

Supervised	Reinforcement
Learns from examples	Learns from interaction
Feedback is usually immediate	Feedback can be delayed
Example: Spam detection	Example: Game-playing AI



Natural Language Processing (NLP) Models

Definition (10 Marks Answer)

1. Introduction

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) that enables computers to understand, interpret, process, and generate human languages such as English and Hindi. NLP models are computational models designed to perform language-related tasks.

2. Definition

NLP models are mathematical or computational models that analyse human language and learn or apply linguistic patterns to perform tasks such as text classification, language translation, sentiment analysis, and text generation.

3. Types of NLP Models

1. Rule-Based Models: These models use predefined linguistic rules, dictionaries, and grammar to process language.
Example: Identifying questions using predefined rules.
2. Statistical Models: These models use probability and statistical methods to analyse language patterns and predict outcomes.
Example: An N-gram model predicts the next word based on previous words.
3. Machine Learning Models: These models learn language patterns from training data and use them to make predictions.
Examples: Naive Bayes, Logistic Regression and Support Vector Machine (SVM).
4. Deep Learning Models: These models use neural networks to learn complex language patterns and contextual relationships.
Examples: RNN, LSTM and Transformer.

4. Working of NLP Models

NLP models generally work through the following steps:

1. Input: Receive text from the user.
2. Preprocessing: Clean and prepare the text.
3. Tokenization: Divide text into smaller units called tokens.
4. Representation: Convert tokens into numerical representations.
5. Processing: Apply rules or a trained model to analyse the text.
6. Output: Generate a prediction, classification, translation, or response.

5. Applications of NLP Models

- Chatbots: Answer user questions.
- Machine Translation: Translate languages.
- Sentiment Analysis: Identify positive and negative opinions.
- Spam Detection: Identify unwanted emails.
- Text Classification: Categorize documents.
- Text Generation: Generate meaningful text.

6. Advantages

- Automates language-processing tasks.
- Processes large amounts of text efficiently.
- Improves human-computer interaction.
- Supports multilingual communication.

7. Limitations

- May struggle with ambiguity and sarcasm.
- Performance depends on data quality and model design.
- Advanced models may require significant computational resources.

Text Classification — 10 Marks Answer

1. Definition

Text Classification is a Natural Language Processing (NLP) technique in which a computer automatically assigns predefined categories or labels to a text based on its content.
For example, an email system classifies emails as Spam or Not Spam.

2. Types of Text Classification

1. Binary Classification: Classifies text into two categories, such as spam or not spam.
2. Multiclass Classification: Classifies text into one of several categories, such as sports, politics, business, or entertainment.
3. Multilabel Classification: Assigns multiple labels to a single text, such as Technology and Business.
4. Sentiment Classification: Identifies opinions as positive, negative, or neutral.

3. Working of Text Classification

Text classification generally involves the following steps:

1. Data Collection: Collect text data for classification.
2. Preprocessing: Remove unnecessary symbols and clean the text.
3. Tokenization: Divide text into smaller units called tokens.
4. Feature Extraction: Convert text into numerical representations using techniques such as Bag of Words or TF-IDF.
5. Model Training: Train a machine learning model using labelled examples.
6. Classification: Predict the category of new text.

4. Algorithms Used

- Naive Bayes: Uses probability to classify text.
- Logistic Regression: Predicts the probability of a text belonging to a class.
- Support Vector Machine (SVM): Separates classes using a decision boundary.
- Decision Tree: Classifies text using a sequence of decision rules.
- Neural Networks: Learn complex patterns in text data.

5. Example

Consider the following customer reviews:

Text	Classification
"This product is excellent."	Positive
"The service is very bad."	Negative
"The product arrived today."	Neutral

Here, the text classification model analyses each review and assigns the appropriate sentiment category.

6. Applications

1. Email spam detection.
2. Sentiment analysis of customer reviews.
3. News article categorization.
4. Social media content analysis.
5. Document organization.
6. Customer feedback analysis.

7. Advantages

- Saves time by automating text analysis.
- Processes large amounts of text efficiently.
- Improves organization of documents.
- Supports faster decision-making.

8. Limitations

- May misunderstand sarcasm and ambiguous sentences.
- Requires good-quality training data.
- Biased data may produce incorrect classifications.
- New categories may require additional training.

Structural Grammars in NLP — 10 Marks Answer

1. Definition

Structural Grammar is a set of grammatical rules used in Natural Language Processing (NLP) to describe how words combine to form phrases and sentences. It helps computers analyse

the grammatical structure of a sentence and understand the relationships between its components.
Example: "The boy eats an apple."
This sentence contains a noun phrase "The boy" and a verb phrase "eats an apple."

2. Components of Structural Grammar

The main components are:

1. Sentence (S): A complete sentence.
2. Noun Phrase (NP): A group of words that functions as a noun, e.g., "the boy."
3. Verb Phrase (VP): A group of words containing a verb, e.g., "eats an apple."
4. Noun (N): Names a person, place, or thing, e.g., boy, apple.
5. Verb (V): Expresses an action or state, e.g., eat, run.
6. Determiner (Det): Specifies a noun, e.g., the, an, a.

3. Context-Free Grammar (CFG)

Context-Free Grammar is a commonly used grammar formalism in NLP. It defines rules called production rules that specify how a sentence can be formed from smaller components.
Example of grammar rules:
S → NP VP
NP → Det N
VP → V NP
Det → the | an
N → boy | apple
V → eats
Here, the symbol → means "can be replaced by" or "can be expanded into."
Using these rules, the sentence "The boy eats an apple" can be represented as:

- S → NP VP
- NP → Det N → The boy
- VP → V NP → eats an apple

Thus, the grammar explains how the words combine to form a grammatically structured sentence.

4. Types of Structural Grammar

1. Context-Free Grammar (CFG): Uses production rules to define sentence structures. Each rule replaces a single non-terminal symbol.
2. Phrase Structure Grammar: Describes how words combine into phrases and larger sentence structures.
3. Dependency Grammar: Represents grammatical relationships between words, such as subject, verb, and object.

5. Importance of Structural Grammar in NLP

- Helps identify sentence structure.
- Supports parsing and syntactic analysis.
- Helps detect grammatical errors.
- Supports machine translation.
- Improves language understanding systems.

6. Applications

1. Grammar-checking software.
2. Machine translation systems.
3. Chatbots and virtual assistants.
4. Question-answering systems.
5. Natural language understanding.

7. Advantages

- Provides a systematic representation of sentence structure.
- Helps computers analyse relationships between words.
- Makes grammatical analysis easier.

8. Limitations

- Complex sentences may require many grammar rules.
- Ambiguous sentences may have multiple valid structures.
- Simple grammar rules may not capture all real-world language patterns

1. Parsing in NLP — 10 Marks Answer

1. Definition

Parsing is a process in Natural Language Processing (NLP) that analyses the grammatical structure of a sentence according to predefined grammar rules. It identifies phrases, words, and their syntactic relationships.
Example: In the sentence "The boy eats an apple," parsing identifies "The boy" as the subject, "eats" as the verb, and "an apple" as the object.

2. Types of Parsing

1. Top-Down Parsing: Starts from the start symbol (S) and applies grammar rules to generate the sentence structure.
2. Bottom-Up Parsing: Starts with individual words and combines them into phrases until the complete sentence structure is formed.
3. Constituency Parsing: Divides a sentence into grammatical units such as noun phrases (NP) and verb phrases (VP).

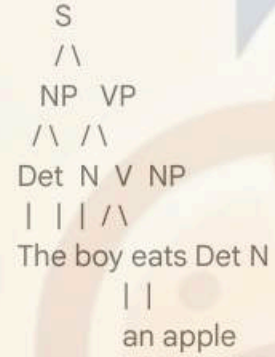
4. Dependency Parsing: Identifies relationships between words, such as subject–verb and verb–object relationships.

3. Working of Parsing

Parsing generally involves the following steps:

1. Input: Receive a sentence.
2. Tokenization: Divide the sentence into individual words.
3. Grammar Application: Apply grammatical production rules.
4. Structure Analysis: Identify phrases and relationships between words.
5. Parse Tree Generation: Represent the grammatical structure in a tree format.

Example sentence: "The boy eats an apple."



4. Applications of Parsing

- Machine translation.
- Grammar checking.
- Chatbots and virtual assistants.
- Question-answering systems.
- Natural language understanding.
- Information extraction.

5. Advantages

- Helps identify sentence structure.
- Improves grammatical analysis.
- Helps determine relationships between words.
- Supports various NLP applications.

6. Limitations

- Ambiguous sentences may have multiple parse trees.
- Complex sentences are difficult to analyse.
- Incorrect grammar rules may produce incorrect structures.

2. Perception: Image Formation — 10 Marks Answer

1. Definition

Image Formation is the process by which a camera or visual sensor captures light reflected or emitted by objects and converts it into an image. In Artificial Intelligence, image formation helps computer vision systems obtain visual information about the environment.

2. Basic Principle of Image Formation

Image formation occurs when light from a scene enters a camera through a lens and falls on an image sensor.
The main components involved are:

1. Light Source: Provides illumination.
2. Object: Reflects or emits light.
3. Lens: Focuses light onto the sensor.
4. Image Sensor: Converts incoming light into electrical signals.
5. Image Processing System: Converts sensor data into a digital image for analysis.

3. Working of Image Formation

1. Light illuminates an object.
2. The object reflects light toward the camera.
3. The camera lens focuses the light.
4. The image sensor captures the light intensity.
5. The sensor converts the captured information into digital pixel values.
6. A computer vision system processes the image to identify objects or extract useful information.

4. Important Concepts

1. Pinhole Camera Model: A simplified camera model in which light passes through a small aperture to form an image.
2. Projection: Converts three-dimensional scene points into two-dimensional image coordinates.
3. Resolution: The number of pixels used to represent an image.
4. Brightness and Illumination: The amount of light affects the intensity and appearance of an image.
5. Colour: Colour images commonly represent each pixel using red, green, and blue (RGB) values.

5. Applications

- Face and object recognition.
- Autonomous vehicles.
- Medical image analysis.
- Robotics and navigation.
- Security and surveillance.
- Industrial quality inspection.

6. Advantages

- Enables machines to obtain visual information.
- Supports automatic object detection.
- Helps robots understand their surroundings.

7. Limitations

- Poor lighting can reduce image quality.
- Shadows and reflections can affect recognition.
- Camera angle and occlusion may hide important objects.

3. Perception: Image Operations — 10 Marks Answer

1. Definition

Image Operations are techniques used to modify, enhance, transform, or analyse digital images. In AI and computer vision, these operations help improve image quality and extract useful information for further processing.

2. Types of Image Operations

1. Image Enhancement: Improves image appearance, brightness, or contrast.
Example: Making a dark image brighter.
2. Image Filtering: Removes noise or highlights specific image features.
Example: Applying a Gaussian filter to reduce noise.
3. Image Transformation: Changes the position, size, or orientation of an image.
Examples: Rotation, scaling, and translation.
4. Image Segmentation: Divides an image into meaningful regions or objects.
Example: Separating a person from the background.
5. Edge Detection: Identifies boundaries where image intensity changes sharply.
Example: Detecting the outline of a building.

6. Thresholding: Converts an image into regions based on pixel intensity values.
Example: Separating black text from a white background.
7. Morphological Operations: Process image shapes, particularly in binary images.

- Erosion: Shrinks foreground regions.
- Dilation: Expands foreground regions.

3. Working of Image Operations

1. Capture the digital image.
2. Apply preprocessing to improve image quality.
3. Use suitable filtering or transformation techniques.
4. Detect important features, edges, or regions.
5. Extract information for recognition or classification.
6. Produce the processed image or analysis result.

4. Applications

- Face recognition.
- Medical image processing.
- Object detection.
- Autonomous driving.
- Satellite image analysis.
- Industrial inspection.

5. Advantages

- Improves image quality.
- Reduces unwanted noise.
- Makes important features easier to detect.
- Supports accurate computer vision tasks.

6. Limitations

- Excessive processing may remove useful details.
- Different images may require different operations.
- Some algorithms require significant computing resources.